HTML Quick Styler

An Interactive Tool for Generating Styled HTML Pages

STUDENT NAME :

1. T. MARYAM SHAHFIYA

2. S. DEVADHARSHINI

3. J. KANIMOZHI INNASIYAMMAL

4. P. MAHESHWARI

5. M.POOJA

REGISTER NUMBER:

1. 23126ER011

2. 23126ER003

3. 23126ER008

4. 23126ER010

5. 23126ER014

INSTITUTION: SRI ADI CHUNCHANAGIRI WOMEN'S COLLEGE, CUMBUM

DEPARTMENT : B.Sc COMPUTER SCIENCE

DATE OF SUBMISSION : 06/03/2026

GITHUB REPOSITORY LINK : https://github.com/Maryam234-am/NM_HTML-QUICK-STYLER_MARYAM-SHAHFIYA.git

HTML Quick Styler

An Interactive Tool for Generating Styled HTML Pages

PROJECT DESCRIPTION

HTML Quick Styler is a Python-based web application developed using **Gradio**, designed to enable users to create fully styled HTML pages quickly and efficiently without any prior coding knowledge. Leveraging the **Gradio Blocks API**, the project organizes interactive components such as textboxes, dropdowns, buttons, and live HTML previews into a clean, responsive layout using tabs, rows, and columns. Users can enter a **page title** and **description**, select a **style theme**, and generate ready-to-use HTML pages instantly.

The application features **pre-designed template cards** and **color palette boxes**, each accompanied by buttons like “Use Template” or “Get Palette,” which dynamically update input fields and allow users to produce styled pages seamlessly. Generated HTML is displayed both as **code** and in a **live preview**, with the content stored in a **Gradio State** component for easy access across tabs.

The combination of Python logic and Gradio’s interactive interface ensures that the tool is **intuitive, efficient, and beginner-friendly**, making it suitable for students, developers, or anyone looking to rapidly prototype and create professional-looking web pages.

PROJECT SCENARIOS

SCENARIO 1: GENERATING A BASIC WEB PAGE

In this scenario, a user opens HTML Quick Styler, enters a page title and description, selects a style theme, and clicks **Generate HTML**. The application produces a styled HTML page along with its code, which the user can preview live in the **Live Preview** tab, enabling quick creation of web pages.

SCENARIO 2: USING A PRE-DESIGNED TEMPLATE

Here, the user navigates to the **Templates** tab and selects a pre-designed template such as “Portfolio” or “Blog.” The application automatically fills the title and description fields, allowing the user to generate a styled page instantly without manual input, saving time and effort.

SCENARIO 3: APPLYING A COLOR PALETTE

In this scenario, the user chooses a preferred color theme from the **Color Palettes** tab by clicking **Get Palette**. The selected palette updates the page style, allowing users to quickly experiment with different design aesthetics before generating the final HTML.

SCENARIO 4: LIVE PREVIEW OF GENERATED PAGES

After generating a page, the user can go to the **Live Preview** tab and click **Load Preview** to see the fully rendered page within the application. This provides a real-time view of how the page will appear in a browser, ensuring the design meets user expectations.

SCENARIO 5: STEP-BY-STEP GUIDANCE FOR BEGINNERS

Finally, beginner users can follow the **Guide** tab, which provides numbered, step-by-step instructions on using HTML Quick Styler efficiently. This scenario ensures users understand the complete workflow, from entering page details to generating and previewing the final web page.

ARCHITECTURAL OVERVIEW

The HTML Quick Styler application is designed using a **modular and interactive architecture** that separates user interface components, backend logic, and live preview functionality. The main architecture is built using **Gradio Blocks**, which organizes the project into tabs for generating pages, templates, color palettes, live preview, and a step-by-step guide. Each tab contains individual components such as textboxes, dropdowns, buttons, HTML display areas, and state storage, allowing for **dynamic interaction and real-time updates**. The backend logic in Python handles **HTML generation, template autofill, and style application**, while the frontend provides an intuitive and responsive interface for users.

CORE TECHNOLOGIES

The following core technologies are used in the development of **HTML Quick Styler**:

- **Python** – The main programming language used for backend logic, HTML generation, and overall functionality.
- **Gradio** – A Python library used to build interactive web interfaces with ease.
- **Gradio Blocks API** – Used to create a modular layout with **tabs, rows, and c--olumns** for organizing components.
- **HTML & CSS** – Used for dynamically generating styled web pages and live previews.
- **Google Fonts** – Integrated to provide professional typography for the generated pages.

PRE-REQUISITES

SOFTWARE REQUIREMENTS:

- **Python 3.x** (preferably 3.10 or above)
- **Gradio library** (version 3.x)
- Internet connection for Google Fonts integration

KNOWLEDGE PRE-REQUISITES:

- Basic understanding of **HTML and CSS**

- Familiarity with **Python programming**
- Understanding of **web page structure** and design principles

HARDWARE REQUIREMENTS:

- A standard computer or laptop with **at least 4GB RAM**
- Any modern **web browser** (Chrome, Firefox, Edge, etc.)
- Basic internet connectivity to load Google Fonts

PHASE 1: SETTING UP THE DEVELOPMENT ENVIRONMENT

Activity 1.1: Setting up Google Colab

The development begins by opening **Google Colab**, which provides a cloud-based Python environment. Required libraries, including **Gradio**, are installed using `pip install gradio`. This ensures a ready-to-use environment for coding and testing the application.

Activity 1.2: Organizing Project Files

The project files are organized in Google Colab, including Python scripts for HTML generation, template handling, and style management. Proper folder structure ensures maintainability and easy access during development.

PHASE 2: CORE APPLICATION DEVELOPMENT

Activity 2.1: Writing Python Logic

The backend logic is implemented in Python. This includes functions to **generate dynamic HTML**, **apply style themes**, **autofill templates**, and **manage color palettes**. Functions are designed to produce code that can be displayed as well as previewed live.

Activity 2.2: Storing Generated HTML

A **Gradio State component** is used to store the generated HTML, allowing it to be accessed across tabs such as Live Preview, Templates, and Color Palettes.

PHASE 3: GRADIO INTERFACE DEVELOPMENT

Activity 3.1: Designing the UI with Gradio Blocks

The user interface is designed using **Gradio Blocks API**. Tabs are created for **Generate Page**, **Live Preview**, **Templates**, **Color Palettes**, and **Guide**, each containing interactive components like textboxes, dropdowns, buttons, HTML displays, and rows/columns for layout.

Activity 3.2: Adding Interactivity

Interactive elements like **Use Template** and **Get Palette** buttons are connected to Python functions so that user actions dynamically update the input fields and enable HTML generation without manual coding.

PHASE 4: DEPLOYMENT ON GOOGLE COLAB

Activity 4.1: Running Gradio App in Colab

The Gradio interface is launched in Google Colab using `demo.launch(share=True)`, allowing the application to be accessed via a public link. This ensures the project can be tested and used from any device with internet access.

Activity 4.2: Sharing the Application

The public Gradio link is shared with users for testing and demonstration purposes, enabling others to interact with the project without needing to set up the environment locally.

PHASE 5: FUNCTIONAL TESTING

Activity 5.1: Testing HTML Generation

All functionalities, including page generation, template autofill, and color palette application, are tested to ensure correct HTML code output and accurate live preview.

Activity 5.2: UI/UX Testing

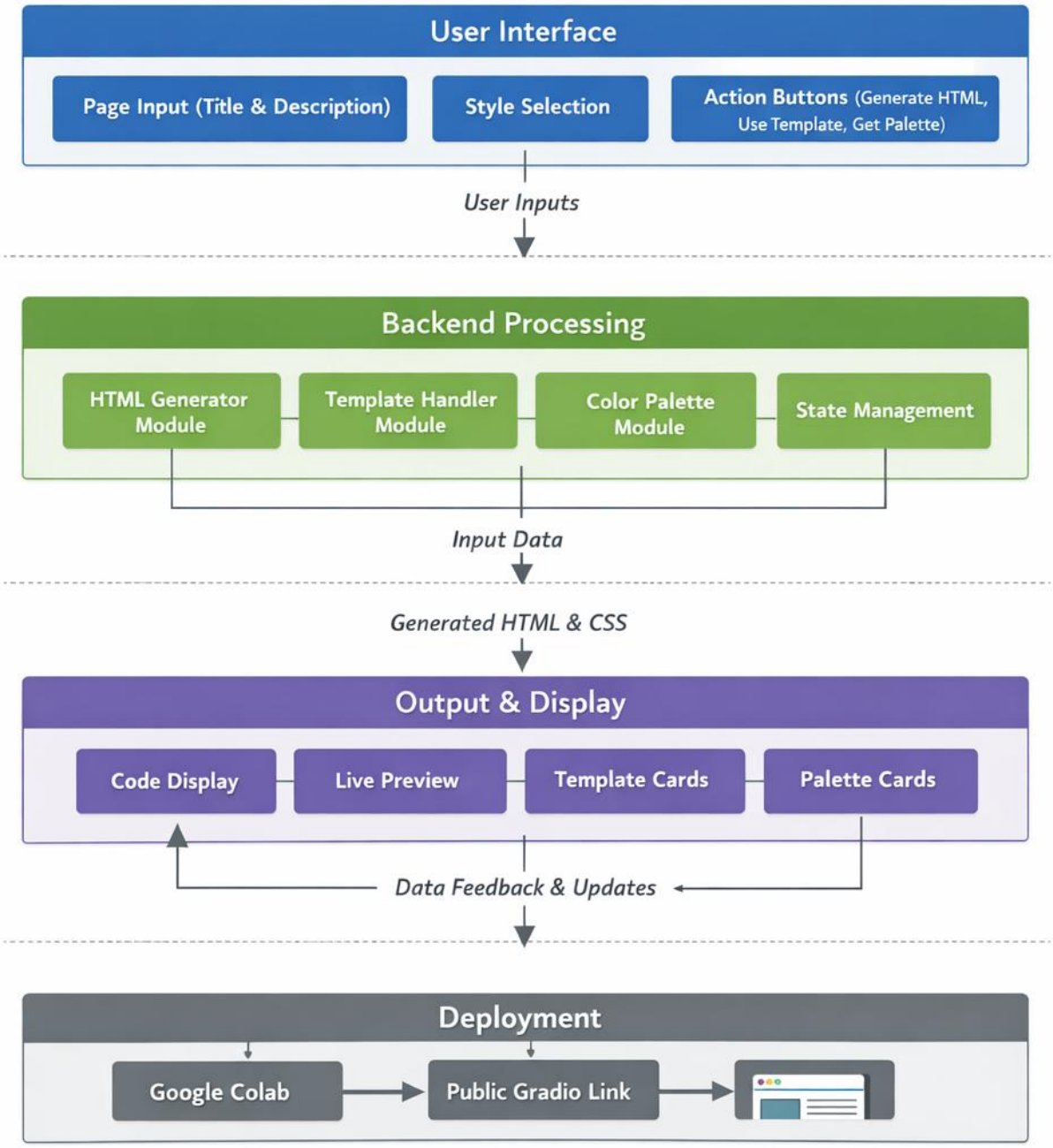
The user interface is tested for responsiveness, alignment, and interactivity. Buttons,

dropdowns, and tabs are checked to ensure a smooth and intuitive experience for end-users.

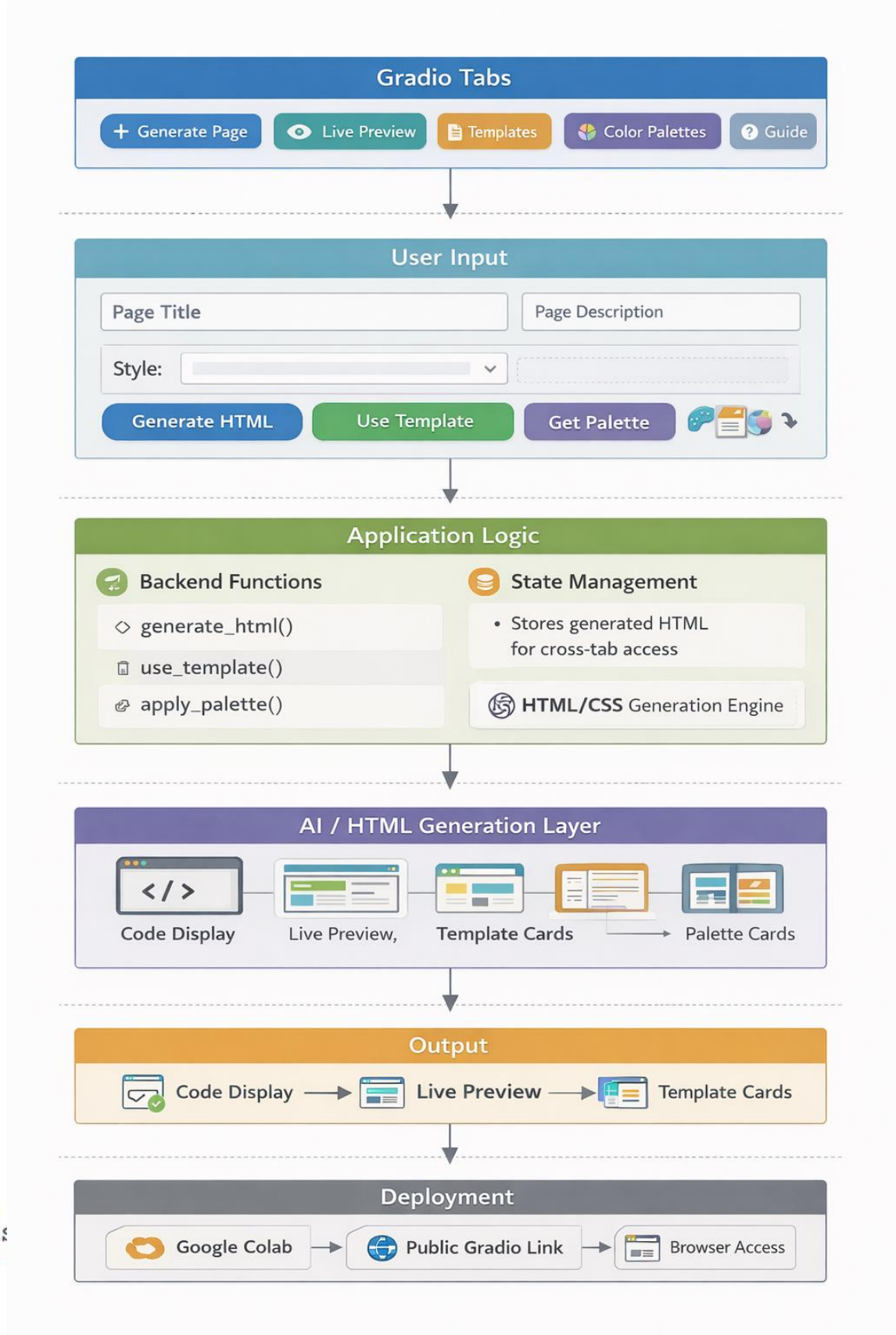
Activity 5.3: Debugging and Refinement

Errors, bugs, and UI inconsistencies identified during testing are corrected, ensuring the final application is fully functional and ready for demonstration or submission.

Technical Architecture Diagram



PROJECT STRUCURE

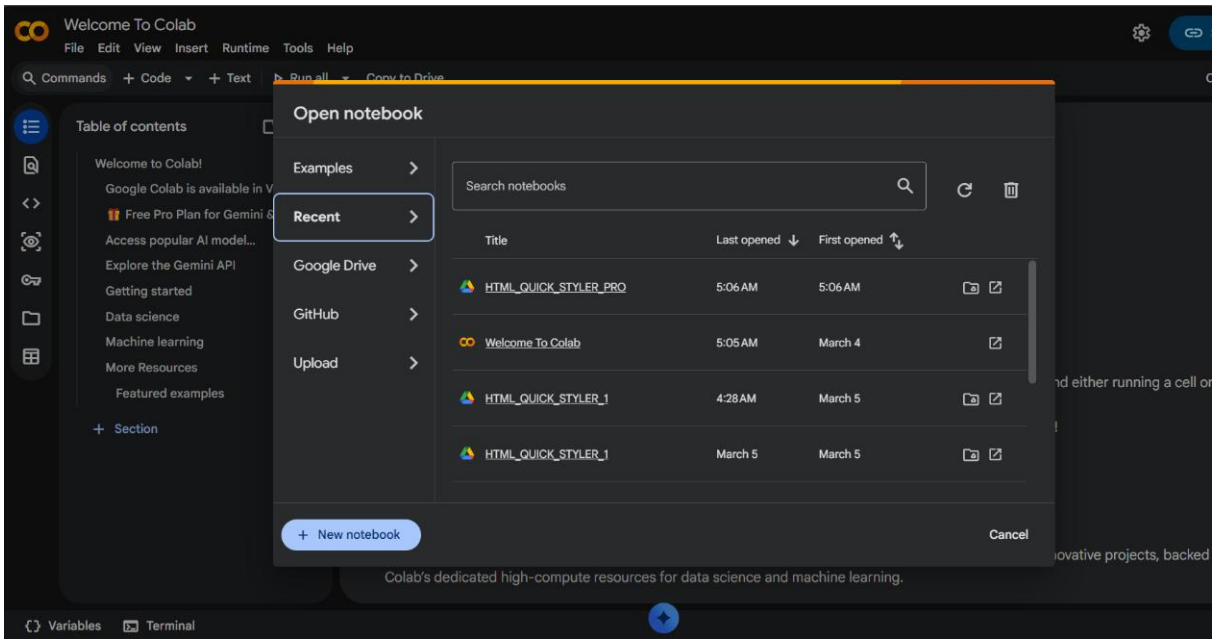


MILESTONE 1: ENVIRONMENT SETUP

Establish the development environment in Google Colab and install the required libraries needed to build the HTML Quick Styler application. This milestone ensures that the system environment is properly configured before implementing the core application features.

ACTIVITY 1.1: SETUP GOOGLE COLAB ENVIRONMENT

- Create a new notebook in **Google Colab**.
- Ensure the **Python runtime environment** is active.
- Verify internet connectivity since the project depends on external libraries.



ACTIVITY 1.2: INSTALL REQUIRED LIBRARIES

- Install the required Python libraries for the project.

```
!pip install gradio transformers torch

Requirement already satisfied: gradio in /usr/local/lib/python3.12/dist-packages (5.50.0)
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: aiofiles<25.0,>=22.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (24.1.0)
Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (4.12.1)
Requirement already satisfied: brotli>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.2.0)
Requirement already satisfied: fastapi<1.0,>=0.115.2 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.133.1)
Requirement already satisfied: ffmpy in /usr/local/lib/python3.12/dist-packages (from gradio) (1.0.0)
Requirement already satisfied: gradio-client==1.14.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.14.0)
Requirement already satisfied: groovy~=0.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.1.2)
Requirement already satisfied: httpx<1.0,>=0.24.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.28.1)
Requirement already satisfied: huggingface-hub<2.0,>=0.33.5 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.5.0)
Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.1.6)
Requirement already satisfied: markupsafe<4.0,>=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.0.3)
Requirement already satisfied: numpy<3.0,>=1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.0.2)
Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.11.7)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from gradio) (26.0)
Requirement already satisfied: pandas<3.0,>=1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.2.2)
Requirement already satisfied: pillow<12.0,>=8.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (11.3.0)
Requirement already satisfied: pydantic<=2.12.3,>=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.12.3)
Requirement already satisfied: pydub in /usr/local/lib/python3.12/dist-packages (from gradio) (0.25.1)
Requirement already satisfied: python-multipart>=0.0.18 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.0.22)
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (6.0.3)
Requirement already satisfied: ruff>=0.9.3 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.15.4)
Requirement already satisfied: safehttpx<0.2.0,>=0.1.6 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.1.7)
Requirement already satisfied: semantic-version~=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.10.0)
Requirement already satisfied: starlette<1.0,>=0.40.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.52.1)
Requirement already satisfied: tomlkit<0.14.0,>=0.12.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.13.3)
```

- Install **Gradio**, which is used to build the interactive web interface.
- Verify the installation by importing the library into the notebook.

ACTIVITY 1.3: IMPORT PROJECT LIBRARIES

- Import the required Python modules for application development.

```
import gradio as gr
```

These libraries allow the application to generate HTML content and create an interactive interface.

MILESTONE 2: CORE APPLICATION DEVELOPMENT

This milestone focuses on developing the core functionality of the HTML Quick Styler application. The main logic responsible for generating dynamic HTML pages, managing styles, and handling template inputs is implemented using Python in Google Colab. The system automatically converts user inputs into a fully structured HTML webpage with styling.

ACTIVITY 2.1: HTML PAGE GENERATION LOGIC

- In this activity, the main function responsible for generating the HTML page dynamically is implemented. The function generate_html() accepts user inputs such as the page title, page description, and selected style theme. Based on these inputs, the function generates a complete HTML document including structure, styling, and content sections.
- The function also defines different styling properties such as background color, text color, button styles, and layout formatting. These styles are applied dynamically depending on the selected theme.

```
import gradio as gr

# HTML generator with dynamic styles
def generate_html(title, description, style):
    bg_color = "#0f172a"
    text_color = "#ffffff"
    btn_color = "#ec4899"
    font = "Poppins"
```

ACTIVITY 2.2: DYNAMIC STYLE AND THEME IMPLEMENTATION

- In this activity, dynamic style themes are implemented to enhance the visual appearance of the generated webpages. Different themes such as **Modern**, **Dark**, and **Minimal** are supported. Each theme applies different background styles, card layouts, and color combinations.
- Conditional logic is used to apply specific styles based on the user's selected theme. This allows the application to generate different visual designs without modifying the core HTML structure.

```python
hero_bg = bg_color
card_bg = "#1e293b"
if style == "modern":
    hero_bg = f"linear-gradient(45deg, {bg_color}, #8b5cf6)"
    card_bg = "#1e293b"
elif style == "dark":
    hero_bg = "#1e293b"
    card_bg = "#111827"
elif style == "minimal":
    hero_bg = bg_color
    card_bg = "#f1f5f9"
```

ACTIVITY 2.3: HTML TEMPLATE CONSTRUCTION

- In this activity, the HTML template structure of the webpage is created inside the Python function. The template includes important components such as the hero section, buttons, content sections, and example template cards.
- Dynamic placeholders are used to insert the user-provided title and description into the webpage automatically. This enables the system to generate a customized webpage layout without manually writing HTML code.

```html
<!DOCTYPE html>
<html>
<head>
<title>{title}</title>
<link href="https://fonts.googleapis.com/css2?family={font.replace(' ', '+')}&display=swap" rel="stylesheet">
<style>{style_css}</style>
</head>
<body>
<div class="hero">
    <h1>{title}</h1>
    <p>{description}</p>
    <button class="btn">Get Started</button>
</div>
```

ACTIVITY 2.4: COLOR PALETTE GENERATOR CLASS

In this activity, a dedicated class named **ColorPaletteGenerator** is created to manage predefined color palettes used in the application. The class stores multiple palettes such as **Modern, Ocean, and Forest**, each containing color properties including primary, secondary, accent, background, surface, and text colors. These palettes help maintain consistent styling across the generated webpage.

The get_palette() method is implemented to retrieve the appropriate palette based on the selected style. If an invalid style is requested, the function automatically returns the **modern palette as the default**, ensuring the application continues to function without errors.

```python
class ColorPaletteGenerator:

    def __init__(self):

        self.palettes = {

            "modern": {
                "primary": "#6366F1",
                "secondary": "#8B5CF6",
                "accent": "#EC4899",
                "background": "#0F172A",
                "surface": "#1E293B",
                "text": "#F8FAFC"
            },

            "ocean": {
                "primary": "#0EA5E9",
                "secondary": "#06B6D4",
                "accent": "#22D3EE",
                "background": "#082F49",
                "surface": "#0C4A6E",
                "text": "#F0F9FF"
            },

            "forest": {
                "primary": "#16A34A",
                "secondary": "#22C55E",
                "accent": "#4ADE80",
```

```python
                "background": "#052E16",
                "surface": "#14532D",
                "text": "#ECFDF5"
            }

        }

    def get_palette(self, style):

        return self.palettes.get(style, self.palettes["modern"])
```

MILESTONE 3: DYNAMIC WEBPAGE GENERATION SYSTEM

ACTIVITY 3.1: HTML PAGE BUILDER CLASS

- In this activity, a class named **HTMLPageBuilder** is implemented to construct different sections of the webpage dynamically. The class contains multiple methods responsible for generating various webpage components such as the Hero Section, About Section, Portfolio Section, and Contact Section.
- Each method returns a structured block of HTML code that represents a specific section of the webpage layout. This modular design allows the system to assemble different sections based on user requirements. By organizing the webpage structure into separate methods, the code becomes more maintainable and reusable.

```python
class HTMLPageBuilder:

    def hero_section(self, title):

        return f"""
        <section class='hero'>
            <h1>{title}</h1>
            <p>Welcome to our website generated with HTML Quick Styler.</p>
            <button>Get Started</button>
        </section>
        """

    def about_section(self):

        return """
        <section class='about'>
            <h2>About Us</h2>
            <p>This page was automatically generated using AI powered HTML Quick Styler.</p>
        </section>
        """

    def portfolio_section(self):

        return """
        <section class='portfolio'>
            <h2>Portfolio</h2>
            <div class='grid'>
                <div class='card'>Project 1</div>
                <div class='card'>Project 2</div>
```

```
            <div class='card'>Project 3</div>
        </div>
    </section>
    """

def contact_section(self):

    return """
    <section class='contact'>
        <h2>Contact Us</h2>
        <input placeholder='Name'>
        <input placeholder='Email'>
        <textarea placeholder='Message'></textarea>
        <button>Send</button>
    </section>
    """
```

ACTIVITY 3.2: CSS GENERATION FOR DYNAMIC STYLING

- In this activity, the **generate_css() function** is implemented to dynamically generate CSS styles for the webpage based on the selected color palette. The function receives the palette dictionary as input and uses the palette values to define styling properties such as background color, text color, hero section gradient, card styling, and layout grids.
- This dynamic CSS generation allows the webpage to adapt visually depending on the selected style theme. By separating the styling logic into a dedicated function, the system ensures better modularity and easier customization.

```
def generate_css(palette):

    return f"""

    body {{
        font-family: Arial, sans-serif;
        background:{palette['background']};
        color:{palette['text']};
        margin:0;
        padding:0;
    }}

    .hero {{
        padding:80px;
        text-align:center;
        background: linear-gradient(45deg, {palette['primary']}, {palette['secondary']});
    }}

    .hero button {{
        padding:12px 25px;
        border:none;
        background:{palette['accent']};
        color:white;
        border-radius:5px;
        font-size:16px;
    }}
```

```
section {{
    padding:60px;
}}

.grid {{
    display:grid;
    grid-template-columns:repeat(3,1fr);
    gap:20px;
}}

.card {{
    padding:30px;
    background:{palette['surface']};
    border-radius:10px;
    text-align:center;
}}

input, textarea {{
    display:block;
    margin:10px 0;
    padding:10px;
    width:300px;
}}

button {{
    cursor:pointer;
}}

"""
```

ACTIVITY 3.3: DYNAMIC WEBPAGE ASSEMBLY LOGIC

- In this activity, the **generate_page() function** is implemented to assemble the final webpage dynamically. The function first retrieves the selected color palette using the ColorPaletteGenerator class. It then creates an instance of the HTMLPageBuilder class to generate different webpage sections.
- Based on the keywords present in the user's description input, specific sections such as Hero, About, Portfolio, and Contact are added to the webpage. The generated HTML sections are then combined with the dynamically generated CSS styles to form a complete HTML document.
- This function plays a crucial role in transforming user input into a fully structured and styled webpage.

```python
def generate_page(title, description, style):

    palette = ColorPaletteGenerator().get_palette(style)

    builder = HTMLPageBuilder()

    html = ""

    description = description.lower()

    if "hero" in description:
        html += builder.hero_section(title)

    if "about" in description:
        html += builder.about_section()

    if "portfolio" in description:
        html += builder.portfolio_section()

    if "contact" in description:
        html += builder.contact_section()

    css = generate_css(palette)

    full_page = f"""
    <html>

    <style>
    {css}
```

```
    </style>

    <body>
    {html}
    </body>

    </html>
    """

    return full_page
```

MILESTONE 4: GRADIO INTERFACE DEVELOPMENT

ACTIVITY 4.1: GRADIO INTERFACE INITIALIZATION

- In this activity, the main user interface of the application is created using the **Gradio Blocks framework**. The Blocks interface allows developers to design structured and interactive web applications by organizing components such as textboxes, buttons, dropdown menus, and tabs.

- The application interface is initialized using gr.Blocks() with a soft theme to provide a modern visual appearance. The project title and description are displayed at the top of the interface using Markdown components.
- This initialization forms the base structure for the entire user interface of the HTML Quick Styler application.

```python
# Template autofill
def use_template(template):
    if template == "portfolio":
        return "My Portfolio", "Welcome to my portfolio where I showcase my projects."
    elif template == "startup":
        return "Startup Landing Page", "Discover our innovative product designed to change the future."
    elif template == "blog":
        return "My Personal Blog", "Thoughts, tutorials, and stories from my journey."
    return "", ""

with gr.Blocks(theme=gr.themes.Soft()) as demo:
    gr.Markdown("# 🎨 HTML Quick Styler Pro")
    gr.Markdown("Generate styled HTML pages with code output and live preview.")

    html_state = gr.State("")

    with gr.Tabs():
```

ACTIVITY 4.2: GENERATE PAGE TAB IMPLEMENTATION

- In this activity, the **Generate Page tab** is implemented to collect the required inputs from the user for webpage generation. The tab includes input fields for entering the webpage title and description, as well as a dropdown menu for selecting the desired style theme.
- A button labeled **Generate HTML** is used to trigger the webpage generation process. When clicked, the generate_html() function processes the inputs and produces the HTML code, which is then displayed in the output section.

```python
# Generate Page Tab
with gr.Tab("Generate Page"):
    title = gr.Textbox(label="Page Title")
    description = gr.Textbox(label="Page Description")
    style = gr.Dropdown(["modern","dark","minimal"], value="modern", label="Style Theme")

    generate_btn = gr.Button("✨ Generate HTML")
    output_code = gr.Code(label="Generated HTML/CSS Code", language="html")

    generate_btn.click(
        lambda t, d, s: (generate_html(t,d,s), generate_html(t,d,s)),
        inputs=[title, description, style],
        outputs=[output_code, html_state]
    )
```

ACTIVITY 4.3: LIVE PREVIEW TAB IMPLEMENTATION

- In this activity, the **Live Preview tab** is developed to allow users to visualize the generated webpage directly within the application. A **Load Preview** button is provided, which loads the generated HTML content and displays it in the preview component.
- This feature allows users to instantly see how their generated webpage will appear in a browser environment.

```python
# Live Preview Tab
with gr.Tab("Live Preview"):
    gr.Markdown("### Click the button below to load the generated page")
    load_btn = gr.Button("Load Preview")
    live_preview = gr.HTML(label="Website Preview")
    load_btn.click(lambda html: html, inputs=html_state, outputs=live_preview)
```

ACTIVITY 4.4: TEMPLATES TAB IMPLEMENTATION

- In this activity, the **Templates tab** is implemented to provide predefined webpage templates that users can quickly apply. Each template is displayed as a card describing the purpose of the template, such as Portfolio, Startup, or Blog.
- Below each card, a **Use Template** button is provided. When the button is clicked, predefined values are automatically filled into the title and description input fields, helping users generate webpages more quickly.

```
# Templates Tab
with gr.Tab("Templates"):
    gr.Markdown("### Quick Templates")
    with gr.Row():
        for name, desc, key in [("Portfolio","Showcase your projects in a clean layout.","portfolio"),
                                ("Startup","Modern landing page for your innovative product.","startup"),
                                ("Blog","Share your thoughts, tutorials, and stories.","blog")]:
            with gr.Column(scale=1):
                gr.HTML(f'<div class="card"><h3>{name}</h3><p>{desc}</p></div>')
                btn = gr.Button("Use Template")
                btn.click(lambda k=key: use_template(k), outputs=[title, description])
```

ACTIVITY 4.5: COLOR PALETTE TAB IMPLEMENTATION

- In this activity, the **Color Palettes tab** is created to allow users to apply predefined style themes to the generated webpage. Each palette is represented as a visual card displaying the palette name and description.
- Below each palette card, a **Get Palette** button is provided. When clicked, the selected palette automatically updates the style dropdown in the Generate Page tab.

```
# Color Palettes Tab
with gr.Tab("Color Palettes"):
    gr.Markdown("### Click 'Get Palette' to apply a style")
    with gr.Row():
        for palette_name, label in [("modern","🎨 Modern<br>Purple gradient modern design."),
                                    ("dark","🌙 Dark<br>Dark UI with minimal highlights."),
                                    ("minimal","⚪ Minimal<br>Clean and simple light theme.")]:
            with gr.Column(scale=1):
                gr.HTML(f'<div class="palette">{label}</div>')
                btn = gr.Button("Get Palette")
                btn.click(lambda n=palette_name: n, outputs=style)
```

ACTIVITY 4.6: GUIDE TAB IMPLEMENTATION

- In this activity, a **Guide tab** is implemented to provide step-by-step instructions for users on how to use the application. The guide explains the process of entering webpage details, generating HTML code, and previewing the generated webpage.
- This feature improves the usability of the system by helping users understand how to operate the application effectively.

```
# Guide Tab (UPDATED)
with gr.Tab("Guide"):
    gr.HTML("""
    <div style="background:#1e293b; padding:30px; border-radius:12px; color:white; max-width:600px; margin:auto;">
      <h2 style="text-align:center; margin-bottom:20px;">📖 How to Use HTML Quick Styler</h2>

      <div style="display:flex; align-items:flex-start; margin-bottom:15px;">
        <div style="flex-shrink:0; width:30px; height:30px; border-radius:50%; background:#ec4899; display:flex; align-items:center; justify-content:center; font-we
        <div>Enter your <b>Page Title</b> and <b>Page Description</b>.</div>
      </div>

      <div style="display:flex; align-items:flex-start; margin-bottom:15px;">
        <div style="flex-shrink:0; width:30px; height:30px; border-radius:50%; background:#ec4899; display:flex; align-items:center; justify-content:center; font-we
        <div>Choose a <b>Style Theme</b> from the dropdown.</div>
      </div>

      <div style="display:flex; align-items:flex-start; margin-bottom:15px;">
        <div style="flex-shrink:0; width:30px; height:30px; border-radius:50%; background:#ec4899; display:flex; align-items:center; justify-content:center; font-we
        <div>Click the <b>Generate HTML</b> button to create your page.</div>
      </div>
```

```
      <div style="display:flex; align-items:flex-start; margin-bottom:15px;">
        <div style="flex-shrink:0; width:30px; height:30px; border-radius:50%; background:#ec4899; display:flex; align-items:center; justify-content:center; font-we
        <div>Copy the <b>Generated HTML/CSS Code</b> from the output box.</div>
      </div>

      <div style="display:flex; align-items:flex-start;">
        <div style="flex-shrink:0; width:30px; height:30px; border-radius:50%; background:#ec4899; display:flex; align-items:center; justify-content:center; font-we
        <div>Go to the <b>Live Preview</b> tab and click <b>Load Preview</b> to view your website live.</div>
      </div>
    </div>
    """)
```

MILESTONE 5: DEPLOYMENT

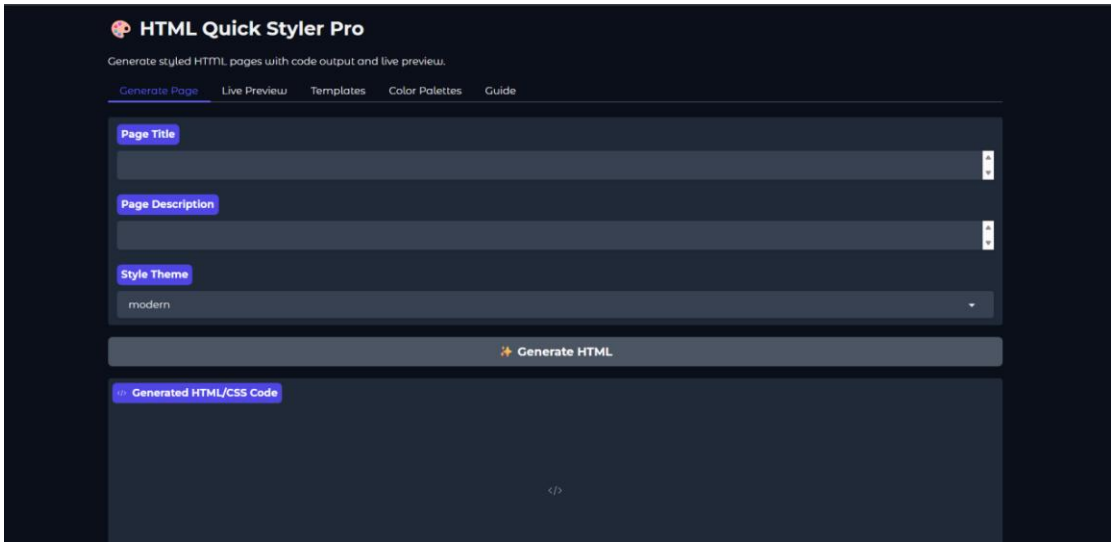
ACTIVITY 5.1: APPLICATION DEPLOYMENT IN GOOGLE COLAB

- In this activity, the HTML Quick Styler application is deployed and executed using **Google Colab**, which provides a cloud-based environment for running Python programs. Google Colab allows developers to run the application without requiring local software installation, making it convenient for testing and demonstration.
- The entire project code is executed within the Google Colab notebook environment. Once the program is run, the Gradio framework automatically creates a web interface for the application.
- This step ensures that the application can be accessed through a browser and interacted with in real time.

```
demo.launch(share=True)
```

```
/tmp/ipykernel_363/4054866721.py:135: DeprecationWarning: The 'theme' parameter in the Blocks constructor will be removed in Gradio 6.0. You will need to pass 'theme' to Blo
  with gr.Blocks(theme=gr.themes.Soft()) as demo:
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://d1cdb8a554c0261eb1.gradio.live
```

The application will be accessible via: https://d1cdb8a554c0261eb1.gradio.live/



MILESTONE 6: TESTING AND VALIDATION

- In this activity, the functional behavior of the HTML Quick Styler application is tested to ensure that all core features operate correctly. Each component of the system, including webpage generation, template selection, palette application, and live preview functionality, is tested using different user inputs.
- The Generate Page feature is tested by entering different titles, descriptions, and style themes to verify that the system correctly produces the corresponding HTML code. The Templates tab is tested by selecting different templates and confirming that the predefined values are automatically inserted into the input fields.
- These tests confirm that the core functionality of the application performs as expected.

ACTIVITY 6.1: FUNCTIONAL TESTING

Functional testing is performed to verify that the core features of the HTML Quick Styler application work correctly. Different inputs are provided to the system to ensure that the application generates the expected HTML output and renders the webpage properly.

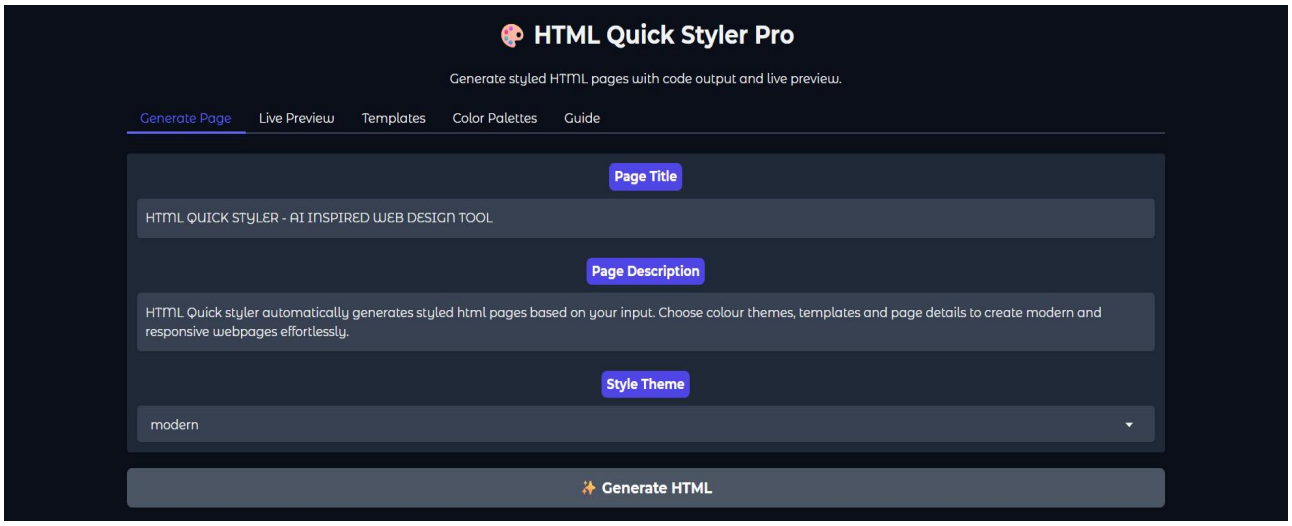
TEST CASE 1: PORTFOLIO WEBSITE GENERATION

Input:

1. **Page Title**: HTML QUICK STYLER - AI INSPIRED WEB DESIGN TOOL
2. **Page Description**: HTML Quick styler automatically generates styled html pages based on your input. Choose colour themes, templates and page details to create modern and responsive webpages effortlessly.
3. **Color Style:** modern

Process:

The user enters the title and description in the Generate Page tab and selects the modern style theme. The Generate HTML button is clicked to produce the HTML code.



Generated HTML/CSS Code

```html
<!DOCTYPE html>
    <html>
    <head>
    <title>HTML QUICK STYLER - AI INSPIRED WEB DESIGN TOOL</title>
    <link href="https://fonts.googleapis.com/css2?family=Poppins&display=swap" rel="stylesheet">
    <style>
    body {
        font-family: 'Poppins', sans-serif;
        background:#0f172a;
        color:#ffffff;
        margin:0;
        padding:40px;
        text-align:center;
    }
    .hero {
        background:linear-gradient(45deg, #0f172a, #8b5cf6);
        padding:60px;
        border-radius:12px;
        color:#ffffff;
    }
    .btn {
        background:#ec4899;
        padding:12px 24px;
        border:none;
        color:white;
        border-radius:6px;
        margin-top:20px;
        font-size:16px;
        cursor:pointer;
    }
```

```
.card {
    background:#1e293b;
    color:#ffffff;
    padding:20px;
    border-radius:10px;
    width:220px;
    height:150px;
    display:flex;
    flex-direction:column;
    justify-content:center;
    transition: transform 0.2s, box-shadow 0.2s;
    text-align:center;
    margin:auto;
}
.card:hover {
    transform: scale(1.05);
    box-shadow: 0px 5px 15px rgba(0,0,0,0.3);
}
.portfolio {
    display:flex;
    flex-wrap:wrap;
    justify-content:center;
    gap:20px;
    margin-top:20px;
}
.palette {
    background:#1e293b;
    color:white;
    padding:20px;
    border-radius:10px;
    text-align:center;
    cursor:pointer;
    transition: transform 0.2s, box-shadow 0.2s;
    width:150px;
    margin:auto;
}
.palette:hover {
    transform: scale(1.05);
    box-shadow: 0px 5px 15px rgba(0,0,0,0.3);
}
```

```html
</style>
</head>
<body>
<div class="hero">
    <h1>HTML QUICK STYLER - AI INSPIRED WEB DESIGN TOOL</h1>
    <p>HTML Quick Styler automatically generated styled html pages based on your input. Choose colour themes, templates and page details to create modern and
    <button class="btn">Get Started</button>
</div>
<div class="section">
    <h2>Example Templates</h2>
    <div class="portfolio">
        <div class="card">
            <h3>Basic Webpage</h3>
            <p>A simple styled webpage layout.</p>
        </div>
        <div class="card">
            <h3>Portfolio Layout</h3>
            <p>Clean design for projects showcase.</p>
        </div>
        <div class="card">
            <h3>Landing Page</h3>
            <p>Modern page for product or service.</p>
        </div>
    </div>
</div>
</body>
</html>
```

Expected Output:

The system generates a styled HTML webpage containing a hero section with the provided title and description along with example portfolio template cards. The generated HTML code is displayed in the output panel.

Result:

The HTML code is generated successfully and displayed in the output section without errors.

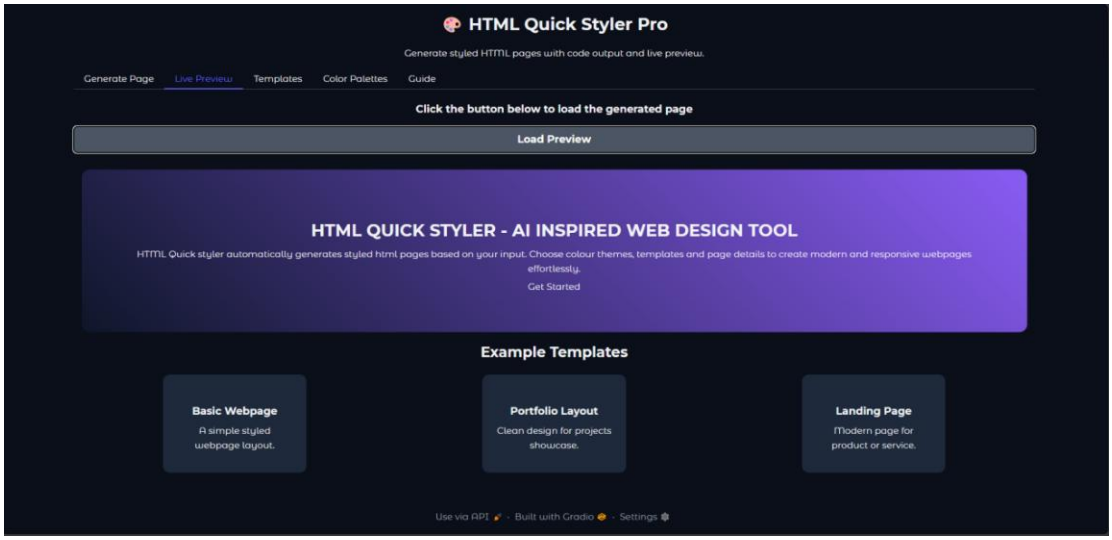
TEST CASE 2: LIVE PREVIEW RENDERING

Input:

Generated HTML webpage

Process:

After generating the HTML code, the user navigates to the Live Preview tab and clicks the **Load Preview** button.



Expected Output:

The generated webpage is rendered inside the preview window, displaying the styled hero section and template cards.

Result:

The webpage preview loads successfully and displays the generated content.

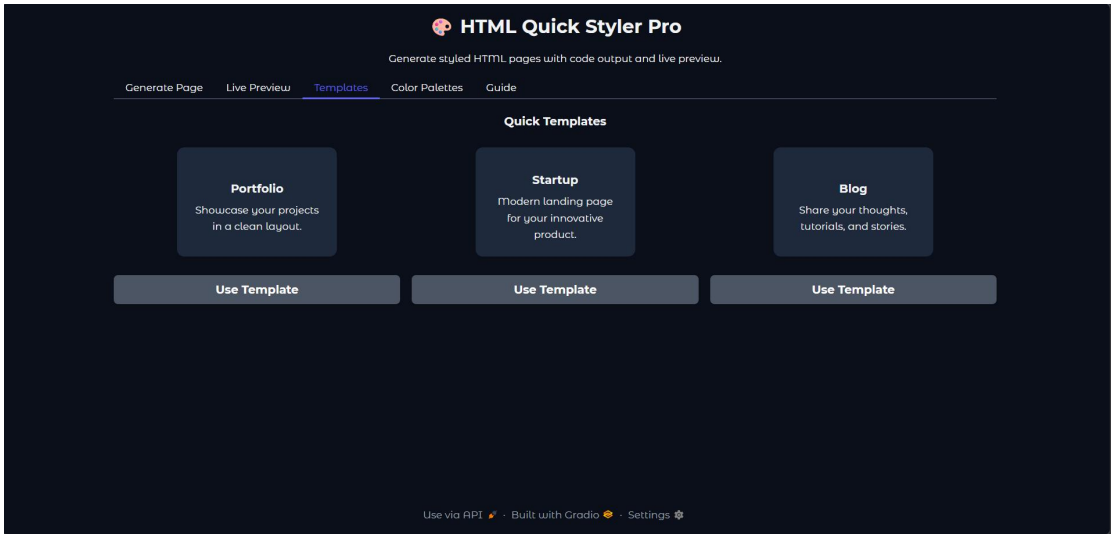
TEST CASE 3: TEMPLATE AUTOFILL FUNCTIONALITY

Input:

• Template Selected: Portfolio Template

Process:

The user navigates to the Templates tab and clicks the **Use Template** button under the Portfolio template card.



Expected Output:

The system automatically fills the Page Title and Page Description fields in the Generate Page tab with predefined portfolio content.

Result:

The template values are successfully inserted into the input fields.

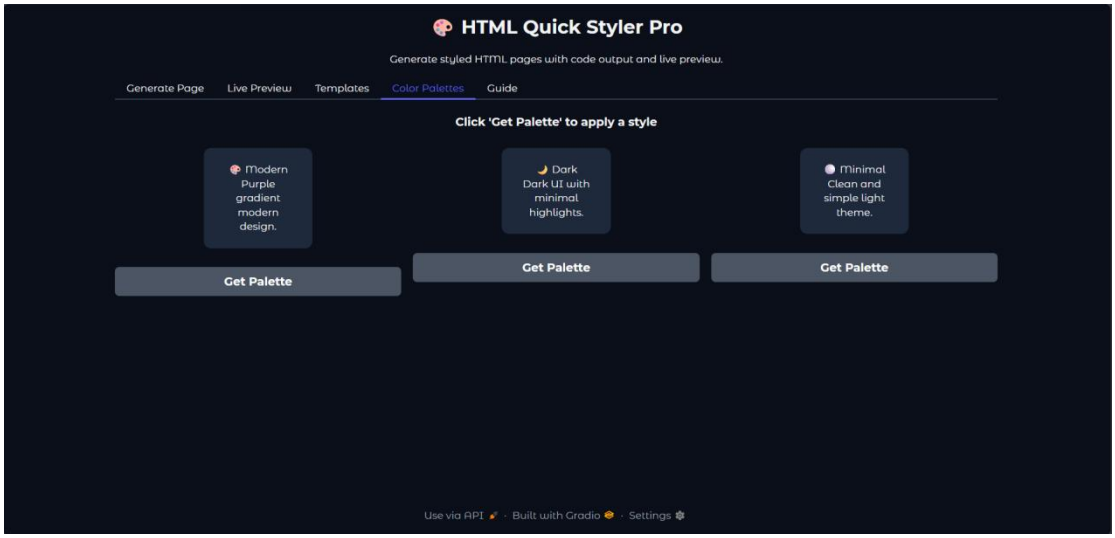
TEST CASE 4: COLOR PALETTE SELECTION

Input:

• Selected Palette: Dark Theme

Process:

The user navigates to the Color Palettes tab and clicks the **Get Palette** button under the Dark palette.



Expected Output:

The style dropdown in the Generate Page tab automatically updates to the selected palette.

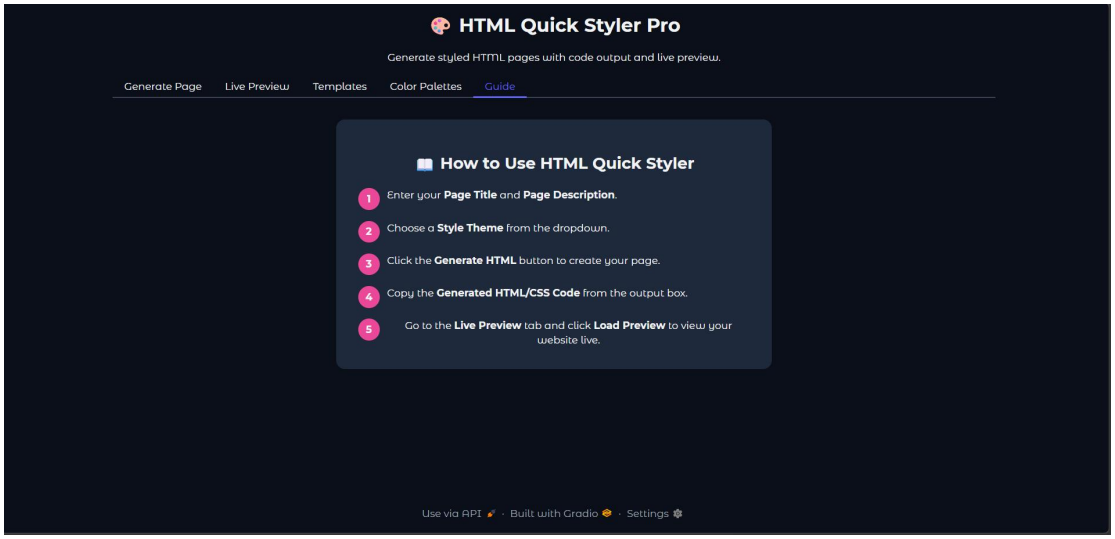
Result:

The style theme is successfully updated to the selected palette.

TEST CASE 5: GUIDE TAB DISPLAY

Input:

User clicks the **Guide Tab** in the Gradio interface.



Expected Output:

The system should display instructions explaining how to use the HTML Quick Styler tool.

The guide should contain steps for entering the title, description, and selecting a style.

Result:

The guide tab successfully displayed the instructions for using the tool.

CONCLUSION

- The **HTML Quick Styler** project was successfully designed and developed to simplify the process of creating stylish HTML webpages. The system allows users to generate structured webpages by simply providing a **title, description, and preferred color style**.
- Through the use of **Python and Gradio**, the project provides an interactive interface that makes webpage generation easy even for users with limited coding knowledge. The tool automatically creates different sections such as **Hero, About, Portfolio, and Contact**, along with appropriate CSS styling to improve the visual appearance of the page.
- The project was tested through various **functional and interface testing methods** to ensure that the system works correctly and generates valid HTML pages. The deployment through **Google Colab with a public URL** allowed users to easily access and test the application.
- Overall, the **HTML Quick Styler** demonstrates how simple inputs can be transformed into structured and visually appealing webpages using automation. This project highlights the usefulness of combining **Python programming, web technologies, and user interface frameworks** to create efficient web development tools